

PRUEBAS TOTALES
EJECUTADAS

117

TASA DE ÉXITO GENERAL

100%

PRUEBAS PASADAS /
FALLADAS

117 / 0

1. Introducción al Ecosistema de Pruebas

Para asegurar la confiabilidad, consistencia y correcto comportamiento de la plataforma humanitaria **Donatón**, se ha implementado un esquema robusto de pruebas automatizadas que abarca tanto el frontend de usuario (desarrollado en React y Vite) como los microservicios distribuidos del backend (desarrollados en Spring Boot).

La estrategia de testing incluye pruebas unitarias y de integración de controladores, pruebas unitarias para servicios de negocio y el mockeo completo de repositorios y comunicaciones externas (como publicación en colas RabbitMQ) para garantizar ejecuciones aisladas e independientes.

2. Métricas y Cobertura del Backend

La compilación de pruebas de backend se realiza mediante el reactor Maven Multi-Módulo a través del comando `./mvnw test`. El suite de pruebas backend consta de **78 casos de prueba unitarios y de integración** que validan el 100% de la funcionalidad de negocio expuesta por los microservicios.

Resumen de Ejecución de Pruebas Backend

Total de Pruebas: 78 ejecutadas con éxito en 15 suites de pruebas integradas. 0 errores, 0 fallos y 0 omitidas. El tiempo acumulado del reactor fue de 31.9 segundos.

Métricas Detalladas por Microservicio

Módulo Microservicio	Clases de Pruebas Evaluadas	Casos	Fallos	Tiempo	Estado
ms-auth	AuthControllerTest, AuthServiceTest	13	0	3.17s	PASSED

Módulo Microservicio	Clases de Pruebas Evaluadas	Casos	Fallos	Tiempo	Estado
ms-donacion	DonacionControllerTest, DonacionServiceTest	10	0	2.94s	PASSED
ms-logistica	LogisticaControllerTest, LogisticaServiceTest	12	0	3.10s	PASSED
ms-matching	MatchingControllerTest, MatchingServiceTest	9	0	2.67s	PASSED
ms-necesidad	NecesidadControllerTest, NecesidadServiceTest	9	0	2.51s	PASSED
ms- notificaciones	NotificacionControllerTest, NotificacionServiceTest	9	0	2.73s	PASSED
ms-usuarios	UsuarioControllerTest, UsuarioServiceTest	13	0	2.69s	PASSED
ms-gateway	GatewayControllerTest	3	0	0.08s	PASSED

3. Métricas y Cobertura del Frontend

El frontend ejecuta sus pruebas de manera aislada utilizando **Vitest** y **React Testing Library** a través del comando `npm run test:coverage`. Las pruebas validan la navegación estructurada, las llamadas y enrutamientos a la API BFF y el comportamiento predecible del diseño por componentes (Composite pattern).

Se ejecutan **39 pruebas en 3 archivos de suite de pruebas**, con una tasa de aprobación del 100% en un tiempo de ejecución de 2.40 segundos.

Reporte de Cobertura de Código (Vitest & V8 Coverage)

El reporte detallado de cobertura de código por archivos y carpetas revela la cobertura sobre las capas esenciales del cliente web:

Archivo / Directorio	% Sentencias	% Ramas	% Funciones	% Líneas	Visualización
Todos los archivos (Global)	39.53%	24.39%	37.50%	43.58%	
src/auth/ProtectedRoute.jsx	100.0%	100.0%	100.0%	100.0%	
src/api/endpoints.js	57.14%	100.0%	53.84%	57.14%	
src/shared/components/index.jsx	13.04%	6.06%	10.00%	15.78%	

Análisis de la Cobertura del Frontend

El componente clave `ProtectedRoute.jsx`, responsable del patrón de enrutamiento proxy-guard que restringe el acceso de usuarios no autorizados, logra un **100% de cobertura** en todas sus sentencias y flujos lógicos. El archivo común de componentes `index.jsx` reporta una cobertura menor (15.78% de líneas), debido a que actualmente las pruebas se centran en verificar exhaustivamente el componente `StatusBadge`, el cual se renderiza con consistencia visual para todos los estados del negocio.

4. Ejemplos de Pruebas Realizadas y Resultados

A continuación se muestran tres ejemplos representativos del ecosistema de pruebas del proyecto, correspondientes a un controlador REST de Spring Boot, un servicio transaccional con publicador de RabbitMQ y un componente visual de React.

Ejemplo 4.1: Prueba de Integración Web - NecesidadControllerTest (Backend)

Este test valida mediante `MockMvc` que el endpoint de creación de necesidades sea accesible, procese datos JSON entrantes y devuelva la estructura DTO esperada.

```
@WebMvcTest(NecesidadController.class)
@AutoConfigureMockMvc(addFilters = false)
public class NecesidadControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private NecesidadService necesidadService;

    @Test
    public void crearShouldReturnNeedResponse() throws Exception {
        NecesidadResponse response = NecesidadResponse.builder()
            .id(1L)
            .descripcion("Comida para damnificados")
            .estado(Necesidad.EstadoNecesidad.ACTIVA)
            .build();

        when(necesidadService.crear(any())).thenReturn(response);

        mockMvc.perform(post("/api/necesidades")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"descripcion\":\"Comida para damnificados\",\"cantidad\":100}"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1L))
            .andExpect(jsonPath("$.descripcion").value("Comida para damnificados"))
            .andExpect(jsonPath("$.estado").value("ACTIVA"));
    }
}
```

Ejemplo 4.2: Prueba de Servicio y Publicador RabbitMQ - DonacionServiceTest (Backend)

Este test unitario valida que la creación de una donación persista el objeto mediante el repositorio y dispare el evento correspondiente hacia la cola de RabbitMQ mediante el publisher configurado.

```
@ExtendWith(MockitoExtension.class)
public class DonacionServiceTest {

    @Mock
    private DonacionRepository donacionRepository;

    @Mock
```

```
private DonacionPublisher donacionPublisher;

@InjectMocks
private DonacionService donacionService;

@Test
public void crearShouldSaveAndPublishDonation() {
    DonacionRequest request = new DonacionRequest();
    request.setDescripcion("Ropa de invierno");
    request.setCantidad(10);

    Donacion savedDonacion = Donacion.builder()
        .id(1L)
        .descripcion("Ropa de invierno")
        .cantidad(10)
        .estado(Donacion.EstadoDonacion.PENDIENTE)
        .build();

    when(donacionRepository.save(any(Donacion.class))).thenReturn(savedDonacion);

    DonacionResponse response = donacionService.crear(request);

    verify(donacionRepository, times(1)).save(any(Donacion.class));
    verify(donacionPublisher, times(1)).publishDonacionCreada(any());
    assertEquals("Ropa de invierno", response.getDescripcion());
}
}
```

Ejemplo 4.3: Prueba de Interfaz y Componentes - StatusBadge.test.jsx (Frontend)

Prueba unitaria en Vitest que evalúa el patrón de composición visual. Comprueba que el componente `StatusBadge` de React asigne dinámicamente las clases CSS correctas asociadas a la severidad del color según el estado inyectado.

```
import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import { StatusBadge } from '../shared/components/index.jsx';

describe('StatusBadge', () => {
  const casos = [
    { status: 'PENDIENTE', expectedClass: 'badge-amber' },
    { status: 'ACTIVO', expectedClass: 'badge-green' },
    { status: 'COMPLETADO', expectedClass: 'badge-blue' },
    { status: 'CANCELADO', expectedClass: 'badge-red' },
  ];

  casos.forEach(({ status, expectedClass }) => {
    it(`renderiza estado "${status}" con clase "${expectedClass}"`, () => {
      render(<StatusBadge status={status} />);
      const badge = screen.getByText(status);
      expect(badge).toBeInTheDocument();
      expect(badge).toHaveClass('badge');
      expect(badge).toHaveClass(expectedClass);
    });
  });

  it('usa badge-gray para estado desconocido', () => {
    render(<StatusBadge status="DESCONOCIDO" />);
    const badge = screen.getByText('DESCONOCIDO');
    expect(badge).toHaveClass('badge-gray');
  });
});
```

5. Conclusiones y Plan de Aseguramiento de Calidad

El ecosistema de pruebas implementado en Donatón garantiza:

1. **No regresión:** Los cambios sobre los microservicios se validan instantáneamente, reduciendo fallos colaterales al actualizar componentes comunes del `ms-common`.
2. **Seguridad Operativa:** Módulos críticos como el control de acceso en frontend y autenticación de backend están cubiertos al 100%, bloqueando vulnerabilidades de autorización de forma temprana.
3. **Ciclos Cortos de Feedback:** Las pruebas unitarias locales y de integración se ejecutan de manera ágil (bajo 32s para el backend y 3s para el frontend), facilitando la adopción de Integración Continua (CI/CD).